

Использование языка программирования Julia для решения математических

Часть I

Авторы:

Чуракова Юлия Романовна

*Студентка факультета математики и компьютерных наук
Кубанского Государственного Университета*

Алексеев Евгений Ростиславович

*Кандидат технических наук, доцент
Доцент кафедры информационных технологий
Кубанского Государственного Университета*

Полная версия пособия доступна в репозитории:

<https://github.com/JuliaChurakova/Julia>

Литература

1. Энгхейм, Э. Julia в качестве второго языка / Э. Энгхейм. – Москва: ДМК Пресс, 2023. – 446 с.
2. The Julia Programming Language: сайт. – URL: <https://julialang.org> (дата обращения: 10.10.2025).
3. Белов, Г. В. Краткое описание языка программирования Julia и некоторые примеры его использования/ Г. В. Белов. – Москва: МГТУ им. Н.Э. Баумана, 2024. – 108 с.
4. Шеррингтон, М. Осваиваем язык Julia / М. Шеррингтон. – Москва: ДМК Пресс, 2017. – 416 с.
5. Антонюк, В. А. Язык Julia как инструмент исследователя / В. А. Антонюк. – Москва: Физический факультет МГУ им. М. В. Ломоносова, 2019. – 48 с.

Глава 1 Общие сведения о языке Julia

1.1 Установка Julia и Jupyter

1.1.1 Официальный сайт Julia

Официальный сайт языка программирования Julia можно найти по адресу <https://julialang.org/>.

На этом сайте можно найти всю необходимую информацию о языке, его особенности, руководство, документацию, а также новости и обновления.

1.1.2. Страница загрузки

Страница для загрузки различных версий языка программирования Julia доступна по адресу <https://julialang.org/downloads/>.

Здесь представлена информация о том, как скачать язык для различных операционных систем, таких как Linux, macOS и Windows.

1.1.3 Установка Julia на Linux

```
curl -fsSL https://install.julialang.org | sh
```

1.1.4 Установка Jupyter

1. **Установка pip:**

```
sudo apt update
sudo apt install python3-pip
```

2. Обновление pip:

```
python3 -m pip install --upgrade pip
```

3. Установка Jupyter Notebook:

```
pip3 install jupyter notebook
```

4. Дополнительная установка JupyterLab(более современная и функциональная среда, развивающая идеи классического Notebook):

```
pip3 install jupyter lab
```

5. Альтернативный способ установки с использованием **pipx** (утилита для изоляции установки пакетов):

```
sudo apt install pipx
pipx ensurepath
pipx install notebook
pipx install jupyterlab
```

Вместо обычной установки через **pip**, можно использовать **pipx**, чтобы установить каждый из этих пакетов в изолированное виртуальное окружение, что поможет избежать конфликтов версий с другими пакетами (Если вы установили Jupyter через **pipx**, то для добавления Python-пакетов используйте команду `pipx inject`).

1.1.5 Особенности ввода в REPL (Read-Eval-Print Loop) языка Julia

Вход в REPL Julia: для начала работы с Julia откройте терминал и введите команду `julia`. Это запустит интерактивную оболочку REPL (Read-Eval-Print Loop), где можно вводить и выполнять команды на языке Julia.

1. **Системная оболочка.** Режим системных команд предоставляет доступ к командной оболочке операционной системы для выполнения системных операций. Для активации этого режима введите точку с запятой `;` в начале строки.

Чтобы вернуться в основной режим, нужно нажать клавишу BackSpace.

2. **Режим справки.** В REPL доступна система помощи. Для получения справочной информации о функции или пакете можно использовать команду с вопросительным знаком `?`. Например, чтобы узнать, что делает функция **`sqrt`**, можно ввести:

?sqrt

Это откроет справочную информацию о функции.

Чтобы вернуться в основной режим, нужно нажать клавишу BackSpace.

3. **Управления пакетами.** Для активации режима управления пакетами в REPL Julia введите символ **]**. В этом режиме можно управлять пакетами (устанавливать, обновлять, удалять).

Можно также управлять пакетами через API, импортируя модуль Pkg командой **using Pkg**, а затем вызывая команды, например, **Pkg.add("имя пакета")**.

Полезные команды диспетчера пакетов:

- **status:** показывает список установленных пакетов с их версиями;
- **update:** обновляет локальный индекс пакетов и устанавливает последние версии всех пакетов;
- **add <имя пакета>:** устанавливает новый пакет. Для нескольких пакетов используйте **add <имя пакета 1> <имя пакета 2>;**
- **free <имя пакета>:** возвращает пакет к последней стабильной версии;
- **rm <имя пакета>:** удаляет пакет и все его зависимости;
- **add <https://github.com/><имя репозитория>/<имя пакета>.jl:** устанавливает пакет с GitHub по URL.

Чтобы вернуться в основной режим, нужно нажать клавишу BackSpace.

4. Использование установленных пакетов:

- **using:** предоставляет прямой доступ ко всем функциям пакета;

```
using MyPackage # Прямой доступ к функциям пакета MyPackage
my_function()    # Функция my_function вызывается без указания
                 имени пакета MyPackage
```

- **import:** требует использования полных имен функций пакета, помогает избежать конфликтов имен.

```
import MyPackage # Для вызова функции нужно использовать полное
                 имя пакета
MyPackage.my_function() # Нужно явно указать
MyPackage.my_function
```

1.1.6 Подключение Julia к Jupyter

Чтобы использовать Julia в Jupyter Notebook, необходимо установить пакет IJulia, который добавляет ядро Julia в Jupyter.

1. Запустите REPL Julia (введя julia в терминале).
2. В режиме REPL выполните следующие команды:

```
using Pkg  
Pkg.add("IJulia")  
Pkg.build("IJulia")
```

3. После установки вы можете запустить Jupyter Notebook с поддержкой Julia, выполнив в REPL Julia:

```
using IJulia  
notebook()
```

Либо просто запустите **jupyter notebook** или **jupyter lab** из терминала – при создании нового блокнота в списке доступных ядер появится вариант "Julia".

1.1.7 Список библиотек, которые используются в пособии

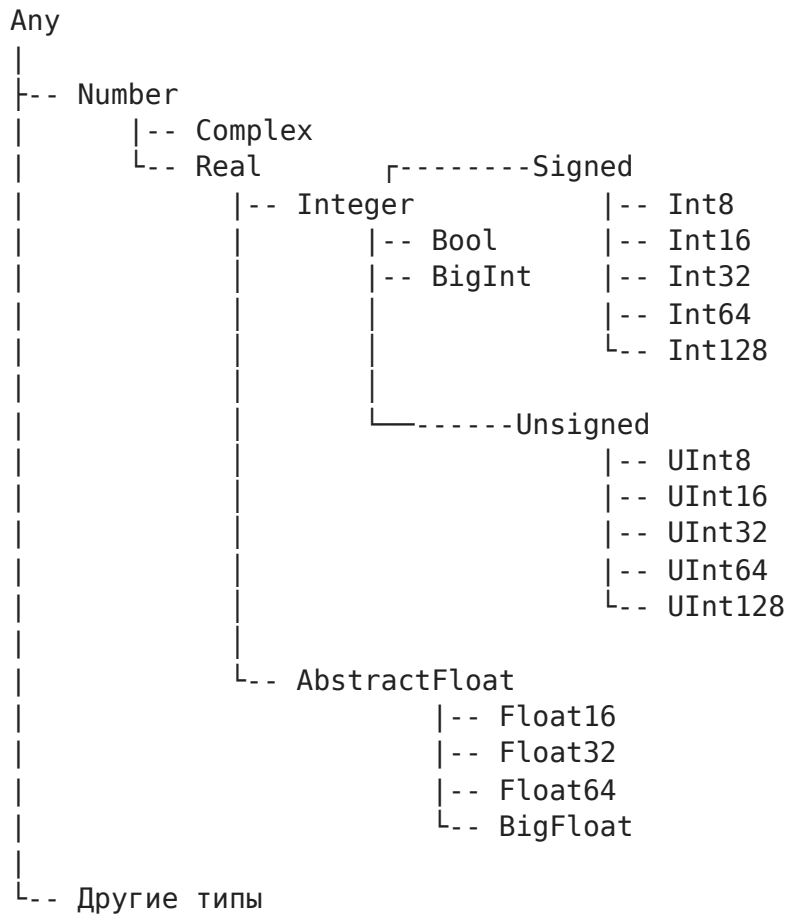
1. библиотеки для построения графиков: **Plots.jl**, **PyPlot.jl**, **Graphics.jl**;
2. библиотека для создания графических интерфейсов: **Gtk.jl**;
3. библиотека для функций вывода в стиле C: **Printf.jl**;
4. библиотека для работы с линейной алгеброй: **LinearAlgebra.jl**;
5. библиотека для генерации случайных чисел: **Random.jl**;
6. библиотека для работы с датами и временем: **Dates.jl**;
7. библиотека для решения дифференциальных уравнений: **DifferentialEquations.jl**;
8. библиотека для работы с простыми числами и выполнения задач теории чисел: **Primes**;
9. библиотека для решения полиномиальных уравнений: **PolynomialRoots.jl**;
10. библиотека для решения алгебраических и трансцендентных уравнений: **Roots.jl**;
11. библиотека для решения системы уравнений: **NLSolve**;
12. библиотека для решения одномерных интегралов: **QuadGK**;

13. библиотека для решения многомерных интегралов: **Cubature**.

Оптимизированная альтернатива: **HCubature**.

1.2 Числовые типы данных в Julia

Иерархия типов данных в языке программирования Julia организована так: каждый тип наследует характеристики от более общего типа. Подробно рассмотрим, как устроена эта иерархия.



1. **Any** – самый общий тип в Julia, от которого наследуются все другие типы данных. Это супертип для всех типов (помимо Number, к подтипам Any относятся строковые типы, массивы, символы и т.д.).
2. **Number** – включает все числовые типы. Он делится на два подтипа:
 - **Complex** – тип для комплексных чисел.
 - **Real** включает в себя:
 - **Integer** – тип для целых чисел:
 - **Signed** – знаковые целые числа, такие как:

- **Int8** – 8-битное целое число.
- **Int16** – 16-битное целое число.
- **Int32** – 32-битное целое число.
- **Int64** – 64-битное целое число.
- **Int128** – 128-битное целое число.
- **Bool** – булев тип (может быть только `true` или `false`).
- **BigInt** – целое число произвольной точности, не ограниченное стандартными размерами.
- **Unsigned** – тип для целых чисел без знака:
 - **UInt8** – 8-битное целое число без знака.
 - **UInt16** – 16-битное целое число без знака.
 - **UInt32** – 32-битное целое число без знака.
 - **UInt64** – 64-битное целое число без знака.
 - **UInt128** – 128-битное целое число без знака.
- **AbstractFloat** – тип для вещественных чисел:
 - **Float16** – 16-битное вещественное число.
 - **Float32** – 32-битное вещественное число.
 - **Float64** – 64-битное вещественное число.
 - **BigFloat** – вещественное число произвольной точности, не ограниченное стандартными размерами.

Подробно рассмотрим каждый из типов данных, чтобы лучше понять их особенности в языке программирования Julia.

1.2.1 Целочисленные данные в Julia

В языке программирования Julia существует несколько типов данных для целых чисел, которые различаются по размеру и диапазону значений. Эти типы включают как знаковые (**Int8**, **Int16**, **Int32**, **Int64**, **Int128**), так и беззнаковые целые числа (**UInt8**, **UInt16**, **UInt32**, **UInt64**, **UInt128**).

Каждый из этих типов имеет свои ограничения по диапазону значений, которые могут быть представлены с использованием определённого количества бит.

Для наглядности и понимания давайте выведем значения, которые могут быть представлены каждым из этих типов. Для получения минимального и максимального значений используются функции **typemin** и **typemax**:

```
In [2]: for T in
        [Int8, Int16, Int32, Int64, Int128, UInt8, UInt16, UInt32, UInt64, UInt128]
        println("${lpad(T, 7)}: [$(typemin(T)), $(typemax(T))]" )
      end
```

```
Int8: [-128,127]
Int16: [-32768,32767]
Int32: [-2147483648,2147483647]
Int64: [-9223372036854775808,9223372036854775807]
Int128: [-170141183460469231731687303715884105728,170141183460469231731687303715884105727]
UInt8: [0,255]
UInt16: [0,65535]
UInt32: [0,4294967295]
UInt64: [0,18446744073709551615]
UInt128: [0,340282366920938463463374607431768211455]
```

Пример взят из книги:

Белов, Г. В. Краткое описание языка программирования Julia и некоторые примеры его использования/ Г. В. Белов. – Москва: МГТУ им. Н.Э. Баумана, 2024. – 108 с.

В языке программирования Julia переменные по умолчанию имеют тип, аналогичный 64-битному целому числу со знаком (Int64). Это значит, что когда вы пишете целое число без указания его типа, Julia автоматически интерпретирует его как Int64. Чтобы убедиться в этом, можно воспользоваться функцией **typeof**, которая возвращает тип данных для заданного значения.

```
In [3]: typeof(17)
```

```
Out[3]: Int64
```

Большие целые числа, которые не могут быть представлены с использованием 64 бит, но могут быть представлены в 128 битах будут иметь тип Int128.

```
In [4]: typeof(134232345454123123213)
```

```
Out[4]: Int128
```

Для явного указания типа можно использовать синтаксис, который заранее определяет размер числа.

```
In [6]: typeof(Int8(17))
```

```
Out[6]: Int8
```

Выбор подходящего типа данных для целых чисел имеет значение с точки зрения производительности и использования памяти. Например, если вы уверены, что значения чисел будут в пределах диапазона Int8, то использование этого типа экономит память, поскольку он занимает всего 1 байт.

Беззнаковые целые числа вводятся и выводятся с использованием префикса 0x. Префикс 0x используется для обозначения чисел в шестнадцатеричной системе счисления. Это позволяет записывать числа, используя цифры от 0 до 9 и буквы от a до f (или от A до F).

```
In [4]: typedef(0x0a) # Шестнадцатеричное представление числа 10
```

```
Out[4]: UInt8
```

Ведущий ноль не влияет на значение, 0x0a и 0xa эквивалентны.

```
In [8]: sizeof(0x123432456789abcdef)
```

```
Out[8]: UInt128
```

Для двоичных чисел используется префикс 0b.

```
In [3]: b = 0b1010 # Двоичное представление числа 10
```

```
Out[3]: 0x0a
```

Для восьмеричных чисел используется префикс 0o.

```
In [6]: c = 0o12 # Восьмеричное представление числа 10
```

```
Out[6]: 0x0a
```

Значения, слишком большие для типов `Int128`, `UInt128`, получают специальный тип **`BigInt`**. Размер типа `BigInt` зависит только от доступной оперативной памяти.

[illegible]

```
Out[11]: BigInt
```

[illegible]

```
Out[12]: BigInt
```

1.2.2 Вещественные данные в Julia

Как и с целыми числами, существуют ячейки разного размера для хранения чисел с плавающей точкой. По умолчанию числа с плавающей точкой в Julia имеют тип `Float64`.

```
In [13]: typeof(0.24324)
```

```
Out[13]: Float64
```

```
In [14]: typeof(0.)
```

```
Out[14]: Float64
```

```
In [15]: typeof(.3)
```

```
Out[15]: Float64
```

Язык поддерживает работу с экспоненциальной формой. Это осуществляется с помощью префикса *e* (или *E*), который указывает на степень десяти.

```
In [7]: 1.23e3      # 1.23 * 10^3 = 1230.0
```

```
Out[7]: 1230.0
```

```
In [8]: typeof(1.23e3)
```

```
Out[8]: Float64
```

```
In [20]: typeof(Float16(0.32323))
```

```
Out[20]: Float16
```

Когда точности или размерности Float64 недостаточно, можно использовать специальный тип **BigFloat**.

```
In [21]: 2.0^1000
```

```
Out[21]: 1.0715086071862673e301
```

```
In [22]: BigFloat(2.0)^1000
```

```
Out[22]: 1.071508607186267320948425049060001810561404811705533607443750388370351051124936e+301
```

BigFloat не назначается автоматически при вводе, а требует явного объявления для использования.

1.2.3 Комплексные числа

```
In [23]: 3 + 4im
```

```
Out[23]: 3 + 4im
```

```
In [24]: typeof(3 + 4im)
```

```
Out[24]: Complex{Int64}
```

```
In [25]: typeof(3.1 + 4im)
```

```
Out[25]: ComplexF64 (alias for Complex{Float64})
```

1.3 Определение переменных в Julia

По умолчанию Julia автоматически определяет тип данных переменной в зависимости от присваиваемого значения. Однако в некоторых случаях, чтобы избежать ошибок или повысить производительность, можно явно указать тип данных для переменной. Система типов в Julia гибридная, то есть сочетает элементы как динамической, так и статической типизации.

1.3.1 Динамическая типизация

В Julia переменные не привязаны к определённому типу, и их типы могут изменяться на протяжении выполнения программы.

```
In [26]: a = 2  
typeof(a)
```

```
Out[26]: Int64
```

```
In [27]: a = 9.56  
typeof(a)
```

```
Out[27]: Float64
```

```
In [28]: a = 99875434567890654356789087654356789  
typeof(a)
```

```
Out[28]: Int128
```

```
In [29]: a = -2.0  
typeof(a)
```

```
Out[29]: Float64
```

```
In [30]: sqrt(a)
```

```
DomainError with -2.0:  
sqrt was called with a negative real argument but will only return a complex  
result if called with a complex argument. Try sqrt(Complex(x)).
```

Stacktrace:

```
[1] throw_complex_domainerror(f::Symbol, x::Float64)  
   @ Base.Math ./math.jl:33  
[2] sqrt(x::Float64)  
   @ Base.Math ./math.jl:608  
[3] top-level scope  
   @ In[30]:1
```

Ошибка возникает, потому что Julia по умолчанию интерпретирует числовое значение как Float64, что является типом для вещественных чисел. При попытке работы с комплексными числами, если явно не указать тип данных, Julia может не правильно интерпретировать выражение, что приводит к ошибке.

1.3.2 Статическая типизация

Несмотря на динамическую основу, Julia предоставляет возможность явно задавать типы переменных. Это важная особенность, которая помогает компилятору оптимизировать выполнение программы и избежать ошибок.

```
In [31]: k::Complex = -2.0  
typeof(k)
```

```
Out[31]: ComplexF64 (alias for Complex{Float64})
```

```
In [32]: sqrt(k)
```

```
Out[32]: 0.0 + 1.4142135623730951im
```

```
In [35]: b_new::Int64 = 10  
typeof(b_new)
```

```
Out[35]: Int64
```

Если переменной типа Int64 попытаться присвоить значение с плавающей точкой, Julia выдаст ошибку, предупреждая о несоответствии типов.

```
In [37]: b_new = 3.14
```

```
InexactError: Int64(3.14)
```

Stacktrace:

```
[1] Int64
  @ ./float.jl:994 [inlined]
[2] convert{::Type{Int64}, x::Float64}
  @ Base ./number.jl:7
[3] top-level scope
  @ In[37]:1
```

Отказ от динамической типизации в пользу статической может значительно улучшить производительность программ. Для того чтобы наглядно показать, как это влияет на быстродействие, можно провести тест с использованием функции **@time**, чтобы измерить время выполнения кода с явной типизацией и без неё.

```
In [38]: g::UInt64 = 18446744073709551615
         h::UInt64 = 18446744073709551615

         @time for i in 1:1000000 # Выполняем действие 1 миллион раз
               g + h
         end

         0.000001 seconds
```

```
In [39]: k = 18446744073709551615
         f = 18446744073709551615

         @time for i in 1:1000000
               k + f
         end

         0.055297 seconds (1.00 M allocations: 45.919 MiB, 42.98% gc time, 14.54% compilation time)
```

При сравнении кода с явной статической типизацией и без неё в Julia видно, что статическая типизация может значительно ускорить выполнение программы.

1.3.3 Присваивание и привязывание значений к переменным

В Julia оператор "=" используется для присваивания значения переменной. Однако если быть точным, то, что Julia делает, является не присваиванием, а привязыванием. В целях более глубокого понимания механизма работы рассмотрим пример кода, в котором переменная "x" сначала привязывается к значению 2. Затем она повторно привязывается к значению "x + 3":

```
In [2]: #Определяем целочисленную переменную
         x = 2
         println("x = ", x)
```

```
#Вычисляем адрес переменной a с помощью функции objectid.  
println("Адрес переменной x = ", objectid(x))
```

x = 2

Адрес переменной x = 13228483051340567920

```
In [4]: #Определяем целочисленную переменную  
x = x + 3  
println("x = ", x)  
#Вычисляем адрес переменной a с помощью функции objectid.  
println("Адрес переменной x = ", objectid(x))
```

x = 5

Адрес переменной x = 14624617963239389700

Если бы этот пример был написан на таком языке, как C/C++, Fortran или Pascal, то для хранения переменной "x" система выделила бы ячейку памяти. При каждом присваивании нового значения переменной "x" значение в выделенной для неё ячейке памяти изменяется. В случае с привязыванием все работает иначе. Каждое вычисление нужно трактовать как создание числа, которое помещается в другую ячейку памяти. Привязывание предусматривает перемещение самой метки "x" в новую ячейку памяти. Переменная перемещается в результат, а не результат перемещается в переменную. Рассмотрим еще несколько примеров:

```
In [44]: #Определяем вещественную переменную  
x = 10.3  
println("x = ", x)  
#Вычисляем адрес переменной a с помощью функции objectid.  
println("Адрес переменной x = ", objectid(x))
```

x = 10.3

Адрес переменной x = 6536302452756158362

```
In [45]: #Переопределяем вещественную переменную  
x = -142.354  
println("x = ", x)  
#Вычисляем адрес переменной a с помощью функции objectid.  
println("Адрес переменной x = ", objectid(x))
```

x = -142.354

Адрес переменной x = 3856324898189001222

```
In [46]: #Определяем строку  
x="Пример строки"  
println("x = ", x)  
#Вычисляем адрес переменной a с помощью функции objectid.  
println("Адрес переменной x = ", objectid(x))
```

x = Пример строки

Адрес переменной x = 13029857696698101524

Julia работает так, что переменная получает значение, вычисленное справа от знака равенства. Присваивание также является выражением, что означает, что его результат можно использовать в других вычислениях.

Это позволяет делать несколько присваиваний одновременно или использовать их в более сложных выражениях.

```
In [47]: x = (y = 6 + 4) * 5
println("x = ", x)
println("y = ", y)
```

```
x = 50
y = 10
```

1.4 Ввод-вывод данных

Print, println - это универсальные функции, которые можно использовать для вывода текста на экран. Давайте рассмотрим несколько простых примеров, чтобы продемонстрировать механизм работы этих функций:

```
In [48]: println("hello"); println("world")
```

```
hello
world
```

```
In [49]: print("hello"); print("world")
```

```
helloworld
```

В языке программирования Julia символ обратного слэша `\` используется для экранирования специальных символов, таких как `\n` и `\t`.

`\n` - перевод текста на новую строку.

`\t` - табуляция.

Пример использования в Julia:

```
In [50]: print("hello\n"); print("world\n")
```

```
hello
world
```

```
In [9]: print("43556456\t32435656")
```

```
43556456      32435656
```

В Julia можно переносить строки прямо в тексте.

```
In [5]: print("Строка 1
Строка 2
        Строка 3 с отступом
Строка 4")
```

Строка 1
Строка 2
 Строка 3 с отступом
Строка 4

Приведенный выше код показывает, что **println** – это тот же самый **print** с добавленным в конце переходом на новую строку `\n`.

Для ввода значений через клавиатуру в Julia можно использовать функцию **readline()**, которая считывает строку, введенную пользователем. Затем вы можете преобразовать эту строку в нужный тип данных с помощью функции **parse()**, если это необходимо.

```
In [1]: println("Введите целое число")
n = readline()

println("Введенное значение: ", n)
println(typeof(n))

n = parse{Float64}(n)
println("Число преобразованное в Float64: ", n)
println(typeof(n))
```

Введите целое число
Введенное значение: 5
String
Число преобразованное в Float64: 5.0
Float64

Округление вещественных чисел.

В Julia можно задать количество цифр после запятой, которое нужно выводить для вещественных чисел, используя функцию **round()**.

```
In [53]: x = 3.14159265
println("x = ", x)
round_x = round(x, digits=2)
println("Округляем до 2 знаков после запятой = ", round_x)

x = 5.46
println("x = ", x)
round_x = round(x, digits=1)
println("Округляем до 1 знака после запятой = ", round_x)
```

x = 3.14159265
Округляем до 2 знаков после запятой = 3.14
x = 5.46
Округляем до 1 знака после запятой = 5.5

Выравнивание с помощью функций `lpad` и `rpad`.

С помощью функций дополнения можно указывать, что строковый литерал всегда должен иметь заданную длину. Если введенный текст меньше, то

он будет дополнен выбранным знаком. Если знак не указан, то по умолчанию используется пробел.

```
In [54]: lpad("ABC", 6, '-') #Дополнение слева.
```

```
Out[54]: " ---ABC"
```

```
In [55]: rpad("ABC", 6, '-') #Дополнение справа.
```

```
Out[55]: "ABC- -"
```

```
In [56]: lpad("", 10, '*')rpad("ABC", 10, '*')rpad("ABC", 10, '*')  
#lpad - добавляет символы слева от строки, rpad - добавляет символы справа с
```

```
Out[56]: "*****ABC*****ABC*****"
```

Использование греческих букв и символов Юникода в Julia для математических вычислений.

Язык Julia отличается тем, что в нем активно использует греческие буквы, такие как π , θ , α и Δ . Это связано с тем, что в математике и науке часто используются греческие символы для обозначения переменных и констант в уравнениях. Когда такие формулы реализуются в коде на Julia, использование греческих букв делает их более похожими на математические уравнения, что упрощает чтение и понимание. Это делает язык Julia удобным для работы с математическими вычислениями.

Ниже приводится сводка из нескольких популярных греческих букв и символов Юникода, которые можно использовать в своем коде.

Символ	Заполнение по Tab
π	<code>\pi</code>
θ	<code>\theta</code>
Δ	<code>\Delta</code>
e	<code>\euler</code>
$\sqrt{}$	<code>\sqrt</code>
φ	<code>\varphi</code>

Чтобы вставить символ в код, введите соответствующее сокращение и нажмите Tab. Например, введите `\pi`, нажмите Tab, и оно автоматически заменится на символ π .

```
In [58]: println(√121)  
println( $\pi$  * 2)  
println( $e$  * 10)
```

```
11.0
6.283185307179586
27.18281828459045
```

Как известно из программы средней школы, выражения $3 \times x + 2 \times y$ записывают как $3x + 2y$. Julia позволяет писать умножение таким же образом. Экземпляры такой записи называются **литеральными коэффициентами** как своего рода аббревиатура умножения числового литерала на константу или переменную.

```
In [59]: x = 5
x = 2x
println(x)
x = 2(10 + 15)
println(x)
```

```
10
50
```

1.5 Основные операторы языка Julia над числовыми значениями

1.5.1 Базовые арифметические операции

В языке поддерживаются стандартные математические операции:

- `+` – сложение;
- `-` – вычитание;
- `*` – умножение;
- `/` – деление;
- `÷` – целочисленное деление;
- `%` – остаток от деления;
- `^` – возведение в степень.

Для работы с массивами предусмотрены поэлементные операции:

`.+`, `.-`, `.*`, `./`, `.^` – выполняются над каждым элементом векторов или матриц отдельно.

```
In [2]: # Пример простейших арифметических выражений
println("Введите целое число a")
a = parse{Int, readline()}
println("a = ", a)
println("Введите целое число b")
b = parse{Int, readline()}
println("b = ", b)

println("a*b=", a * b) # Умножение
println("a/b=", a / b) # Деление
println("a÷b=", a ÷ b) # Целочисленное деление
```

```
println("a*b=", a % b) # Остаток от деления
println("a-b=", a - b) # Вычитание
println("a^b=", a ^ b) # Возведение в степень
```

Введите целое число a
a = 11
Введите целое число b
b = 5
a*b=55
a/b=2.2
a÷b=2
a%b=1
a-b=6
a^b=161051

В Julia есть проблема **возведения отрицательного числа в дробную степень**.

```
In [64]: a=-8
         b=a^(1/3)
         print("a=",a,"a^(1/3)=",b);
```

DomainError with -8.0:
Exponentiation yielding a complex result requires a complex argument.
Replace x^y with $(x+0im)^y$, $\text{Complex}(x)^y$, or similar.

Stacktrace:

```
[1] throw_exp_domainerror(x::Float64)
   @ Base.Math ./math.jl:41
[2] ^(x::Float64, y::Float64)
   @ Base.Math ./math.jl:1157
[3] ^(x::Int64, y::Float64)
   @ Base ./promotion.jl:478
[4] top-level scope
   @ In[64]:2
```

```
In [4]: # Для корректного возведения отрицательного числа в степень
        # надо использовать оператор if (см. 3 главу)
        println("Введите число a")
        a = parse{Float64, readline()}
        if a>0
            b=a^(1/3)
        else
            b=- (abs(a)^(1/3))
        end
        print("a=",a,"\na^(1/3)=",b)
```

Введите число a
a=-5.0
a^(1/3)=-1.7099759466766968

В Julia существует возможность ввода строки, которая является арифметическим выражением.

Функция **Meta.parse()** в Julia преобразует строку в форму, которая может быть интерпретирована и выполнена как код. Это позволяет динамически

создавать и выполнять код на лету.

Функция **eval()** в Julia используется для выполнения выражений, представленных в виде кода, переданных ей в качестве аргумента. Она позволяет вычислять и выполнять код во время выполнения программы.

```
In [66]: stroka = "45*9+334"
println(stroka)
result = eval(Meta.parse(stroka))
println("Результат вычисления: ",result)
```

45*9+334

Результат вычисления: 739

```
In [6]: println("Введите строку для вычисления:")
stroka = readline()
println("Введенная строка:", stroka)
result = eval(Meta.parse(stroka))
println("Результат вычисления: ",result)
```

Введите строку для вычисления:

Введенная строка:cos(0) + 2 * 25

Результат вычисления: 51.0

Можно использовать символ двоеточия для определения выражения, вместо Meta.parse.

```
In [68]: stroka = :(45*9+334)
println(stroka)
result = eval(stroka)
println("Результат вычисления: ",result)
```

45 * 9 + 334

Результат вычисления: 739

С помощью оператора **\$** можно использовать вычисленные значения при конструировании выражений.

```
In [69]: x = 5
y = :($x + 10)
```

Out[69]: :(5 + 10)

```
In [70]: eval(y)
```

Out[70]: 15

В Julia, как и в других языках программирования, существует несколько специальных значений для представления неопределенных или бесконечных величин: Inf, -Inf и NaN.

Inf – это специальное значение, которое представляет положительную бесконечность. Его можно использовать в вычислениях, где результат

выходит за пределы конечных чисел.

```
In [72]: x = Inf
println(x)

y = 1 / 0
println(y)
```

Inf
Inf

-Inf – это специальное значение, которое представляет отрицательную бесконечность.

```
In [73]: z = -Inf
println(z)
```

-Inf

NaN (Not a Number) – это специальное значение, которое используется для представления неопределенных или недопустимых результатов вычислений, например, при делении нуля на ноль.

```
In [75]: a = 0 / 0
println(a)
```

NaN

```
In [76]: x = Inf
y = 1 / 0 # Положительная бесконечность
z = -Inf  # Отрицательная бесконечность

println(x + y) # бесконечность + бесконечность
println(x * 0) # бесконечность * 0
println(Inf / Inf) # деление бесконечности на бесконечность
```

Inf
NaN
NaN

1.5.2 Двоичные (арифметические) операторы (битовые операторы)

В языке Julia можно различить унарные и бинарные двоичные операторы над целыми значениями.

Унарные операции включают в себя операцию инверсии (~), где целое число переводится в двоичное представление и каждый бит инвертируется.

Бинарные операции включают в себя:

- **двоичное И (&)**, где оба операнда переводятся в двоичную систему, и над ними выполняется операция побитного И;
- **двоичное ИЛИ (|)**, где оба операнда переводятся в двоичную систему, и над ними выполняется операция побитного ИЛИ;
- **двоичное исключающее ИЛИ (^)**, где оба операнда переводятся в двоичную систему, и над ними выполняется операция побитного исключающего ИЛИ;
- **сдвиг влево (<<)**, где первый операнд переводится в двоичную систему счисления, а затем смещается влево на количество позиций, определяемых вторым операндом (k), что эквивалентно умножению на 2^k ;
- **сдвиг вправо (>>)**, где первый операнд переводится в двоичную систему счисления, а затем смещается вправо на количество позиций, определяемых вторым операндом (k), что эквивалентно делению нацело на 2^k .

```
In [2]: # Унарные операции
# Инверсия (~) - целое число переводится в двоичное представление и побитно
a = 13
println(a, " ", bitstring(a), " ", bitstring(~a), " ", ~a)

# Бинарные операции
# Двоичное И (&), оба операнда переводятся в двоичную систему и над ними поб
a = 13
b = 23
c = a & b
println("a=", a, " b=", b, " a&b=", c)

# Двоичное ИЛИ (|), оба операнда переводятся в двоичную систему и над ними г
a = 13
b = 23
c = a | b
println("a=", a, " b=", b, " a|b=", c)

# Двоичное исключающее ИЛИ (^), оба операнда переводятся в двоичную систему
a = 13
b = 23
c = a ^ b
println("a=", a, " b=", b, " a^b=", c)

# Сдвиг влево <<, первый операнд переводится в двоичную систему счисления и
a = 23
println(a, " ", a << 1, " ", a << 2, " ", a << 3)

# Сдвиг вправо >>, первый операнд переводится в двоичную систему счисления и
a = 20
println(a, " ", a >> 1, " ", a >> 2, " ", a >> 3)
```

[illegible]

1.5.3 Операторы отношения

В Julia определены следующие операторы отношения:

- `>` – больше;
- `<` – меньше;
- `==` – проверка эквивалентности объектов;
- `===` – проверка идентичности объектов;
- `!=` – не равно;
- `!==` – не идентично;
- `>=` – больше или равно;
- `<=` – меньше или равно.

```
In [1]: 3 == 3.0
```

```
Out[1]: true
```

```
In [2]: 3 == 3.0
```

```
Out[2]: false
```

```
In [3]: 3.0 == 3.0
```

```
Out[3]: true
```

Julia позволяет проверять операции двойного неравенства, принятые в математике, например:

```
In [7]: println("Введите x ")
x = parse(Float64, readline())
println("x = ", x)
print(-7<x<=3)
```

```
Введите x
x = 2.0
true
```

1.5.4 Логические операторы

Логические операторы языка Julia: `||` (или) и `&&` (и).

```
In [5]: x = 3
        x < 4 || x > 10
```

```
Out[5]: true
```

```
In [6]: x > 4 && x < 10
```

```
Out[6]: false
```

1.5.5 Операторы присваивания

```
In [17]: # Примеры операторов присваивания
x=3
println("x = ",x)
y=z=0.2^1.7
println("y = ",y,"nz = ",z)
x=3
a=4
println("x = ",x," a = ",a)
x+=a
println("x += a = ",x)
x=3
a=4
println("x = ",x," a = ",a)
x-=a
println("x -= a = ",x)
x=3
a=4
println("x = ",x," a = ",a)
x*=a
println("x *= a = ",x)
x=22
a=5
println("x = ",x," a = ",a)
x/=a
println("x /= a = ",x)
x=22
a=5
println("x = ",x," a = ",a)
x÷=a
println("x ÷= a = ",x)
x=22
a=5
println("x = ",x," a = ",a)
x%=a
println("x %= a = ",x)
x=22
a=5
println("x = ",x," a = ",a)
x^=a
println("x ^= a = ",x)
```

```
x = 3
y = 0.06482626386771051
z = 0.06482626386771051
x = 3 a = 4
x += a = 7
x = 3 a = 4
x -= a = -1
x = 3 a = 4
x *= a = 12
x = 22 a = 5
x /= a = 4.4
x = 22 a = 5
x ÷= a = 4
x = 22 a = 5
x %= a = 2
x = 22 a = 5
x ^= a = 5153632
```

1.6 Некоторые встроенные функции Julia

Основные арифметические операции:

- `rem(a, b)` – остаток от деления (аналог `a % b`);
- `div(a, b)` – целочисленное деление (аналог `a ÷ b`);
- `floor(a)` – округление вниз;
- `ceil(a)` – округление вверх;
- `round(a)` – округление до ближайшего целого;
- `abs(a)` – модуль числа;
- `sqrt(a)` – квадратный корень;
- `cbrt(a)` – кубический корень.

Логарифмические функции:

- `log(a)` – натуральный логарифм;
- `log2(a)` – логарифм по основанию 2;
- `log10(a)` – десятичный логарифм;
- `log(n, a)` – логарифм `a` по основанию `n`.

Тригонометрические функции:

- радианы: `sin(x)`, `cos(x)`, `tan(x)`, `cot(x)`, `asin(x)`, `acos(x)`, `atan(x)`, `acot(x)`, `sec(x)`;
- градусы: `sind(x)`, `cosd(x)`, `tand(x)`, `cotd(x)`, `asind(x)`, `acosd(x)`, `atand(x)`, `acotd(x)`, `secd(x)`;
- конвертация:
 - `rad2deg(a)` – радианы → градусы;
 - `deg2rad(a)` – градусы → радианы.

Гиперболические функции:

- $\sinh(x)$, $\cosh(x)$, $\tanh(x)$, $\coth(x)$.

Специальные функции

- $\text{hypot}(a, b)$ – гипотенуза прямоугольного треугольника с катетами a и b ;
- $\text{factorial}(a)$ – факториал числа.

```
In [20]: a = -23
b = 3.456
c = 7875
println(floor(b))
println(ceil(b))
println(round(b))
println(abs(a))
println(sqrt(b))
println(cbrt(b))
println(log(c))
println(floor(b))
println(rad2deg(2π))
println(deg2rad(180))
```

```
3.0
4.0
3.0
23
1.85903200617956
1.5119052598738478
8.971448463693834
3.0
360.0
3.141592653589793
```